Universidade da Beira Interior

Engenharia Eletrotécnica e de Computadores

Implementação de uma Arquitetura e Controlador

Microprocessadores

Alexandre Rafael Gravelho Saraiva Nº44485

Docente: Prof. Doutor António Manuel Gonçalves Pinheiro

junho de 2025

Conteúdo

1 Lista de Acrónimos **vi**

2 Resumo **viii**

3 Introdução **1**

4 Fundamentação Teórica **2**
4.1 Arquitetura 2
4.2 Registos – 74LS173 2
4.3 Memória de Registos – 74LS670 3
4.4 Unidade Lógica Aritmética (ALU) 4
4.5 Registo de Flags – 74LS173 5
4.6 Unidade de Controlo – GAL22V10 6

5 Especificação do Projeto **7**
5.1 Objetivos Funcionais 7
5.2 Componentes Principais 7
5.3 Sinais de Controlo 8
5.4 Requisitos de Implementação 8
5.5 Operações Definidas pelo Sistema 8

6 Arquitetura Implementada **9**
6.1 Componentes Utilizados 9
6.2 Esquema da Arquitetura 9
6.3 Ligação entre Módulos 10

7 Controlador **11**
7.1 Fluxograma 11
7.2 Equações Booleanas 13
7.3 Código fonte em WinCupl para a GAL22V10 15

8 Testes e Validação **19**
8.1 Testes à Unidade Lógica Aritmética (ALU) 19
8.2 Estado Atual do Controlador 20
8.3 Considerações Finais sobre os Testes 20

9 Conclusão **21**

Lista de Figuras

1 Diagrama funcional do registo 74LS173. Fonte: Texas Instruments 2
2 Diagrama funcional do 74LS670 (4x4 Register File). Fonte: Texas Instruments 3
3 Diagrama funcional da gal22v10 . Fonte: Lattice Semiconductor Corp. . . 4
4 Diagrama funcional do registo 74LS173. Fonte: Texas Instruments 5
5 Diagrama funcional da gal22v10 . Fonte: Lattice Semiconductor Corp. . . 6
6 Esquema da arquitetura implementada. 10
7 Fluxograma da máquina de estados do controlador digital. 11

1 Lista de Acrónimos

- **ALU** – Unidade Lógica e Aritmética
- **PLD** – Programmable Logic Device
- **FSM** – Finite State Machine
- **LS** – Low-power Schottky (família lógica 74LS)
- **GAL** – Generic Array Logic
- **CLK** – Clock (sinal de relógio)
- **IC** – Integrated Circuit (circuito integrado)
- **MSB** – Most Significant Bit
- **LSB** – Least Significant Bit
- **Z** – Zero (flag de resultado nulo)
- **C** – Carry (flag de transporte aritmético)
- **V** – Overflow (flag de sobrecarga)
- **S** – Sinal (flag de sinal do resultado)
- **UBI** – Universidade da Beira Interior

2 Resumo

O presente relatório descreve o desenvolvimento e a implementação de uma arquitetura, composta por uma Unidade Lógica Aritmética (ALU), Registos de Armazenamento, Memória de Registos e um Controlador. O projeto foi realizado no âmbito da unidade curricular de Microprocessadores da Universidade da Beira Interior.

A arquitetura foi construída com base em circuitos integrados da família 74LS, nomeadamente o 74LS173 e o 74LS670, bem como dispositivos lógicos programáveis (GAL22V10), programados em WinCupl. A ALU, concebida em lógica combinatória, realiza operações aritméticas básicas (soma, subtração, incremento, decremento) e gera flags de estado. O controlador sequencial foi desenvolvido como uma máquina de estados finitos, responsável por coordenar todas as fases do ciclo de execução.

A ALU foi devidamente testada, apresentando resultados corretos. O controlador foi implementado, mas encontra-se ainda por validar completamente em Hardware.

Este trabalho permitiu aplicar conceitos teóricos fundamentais , consolidando conhecimentos na área de microprocessadores e da eletrónica.

3 Introdução

O presente relatório de laboratório insere-se no âmbito da unidade curricular de Microprocessadores.

O principal objetivo deste trabalho é implementar e testar em hardware uma arquitetura baseada numa ALU de 4 operações aritméticas, composta por registos, sinais de controlo e implementar um controlador.

A arquitetura implementada foi desenvolvida em ambiente de hardware, com recurso a circuitos integrados da família 74LS e dispositivos lógicos programáveis (PLDs), nomeadamente a GAL22V10. Esta arquitetura tem como principal função a execução de operações aritméticas básicas (adição, subtração, incremento e decremento), definidas pelos sinais de controlo M_1 e M_0.

O sistema é constituído por:

- Um registo de entrada (R_X), onde são carregados os operandos;
- Uma Unidade Lógica Aritmética (ALU), que recebe os valores A e B e executa uma das quatro operações possíveis;
- Um registo de flags (R_F), que atualiza os indicadores de estado do resultado (zero, sinal, carry, overflow);
- Um registo acumulador (R_A), onde é armazenado o resultado final;
- Uma unidade de controlo que coordena os sinais de escrita e leitura nos registos, utilizando sinais como W_X, W_F, W_{Acc}, D_0, D_1, G_R e G_W.

A comunicação entre os blocos é feita através de um barramento interno, e o funcionamento sequencial do sistema é garantido por uma máquina de estados finitos, programada na GAL. A correta coordenação dos sinais de controlo permite a leitura dos operandos, a execução da operação pretendida na ALU, o armazenamento do resultado e a atualização das flags.

Ao longo deste relatório são apresentados os fundamentos teóricos da arquitetura adotada, o processo de implementação da solução em hardware, bem como os testes realizados para validação funcional do sistema. Este trabalho permitiu a consolidação de conhecimentos sobre ALUs, registos, controlo sequencial e organização interna de microprocessadores, através de uma aplicação prática e completa.

4 Fundamentação Teórica

Nesta secção são apresentados os conceitos teóricos que sustentam o projeto desenvolvido, com ênfase nos componentes efetivamente utilizados: Registos (74LS173), Memória de Registos (74LS670), Unidade Lógica Aritmética (ALU) implementada numa GAL22V10 e Unidade de Controlo também implementada numa GAL22V10.

4.1 Arquitetura

Uma arquitetura é composta por módulos interligados que operam de forma síncrona, permitindo a execução ordenada de operações. Estes módulos incluem Registos de Dados, uma Unidade Lógica Aritmética (ALU), uma Unidade de Controlo Sequencial e, neste caso, uma Memória de Registos que permite armazenar múltiplos operandos. A comunicação entre os blocos é realizada através de um barramento comum, sendo as trocas de dados coordenadas por sinais de controlo gerados pela unidade de controlo principal.

4.2 Registos – 74LS173

Os registos são circuitos síncronos utilizados para armazenamento temporário de dados. No sistema implementado foram utilizados tres circuitos 74LS173, registos D-Type de 4 bits com entradas de controlo de escrita, ativação e clear.

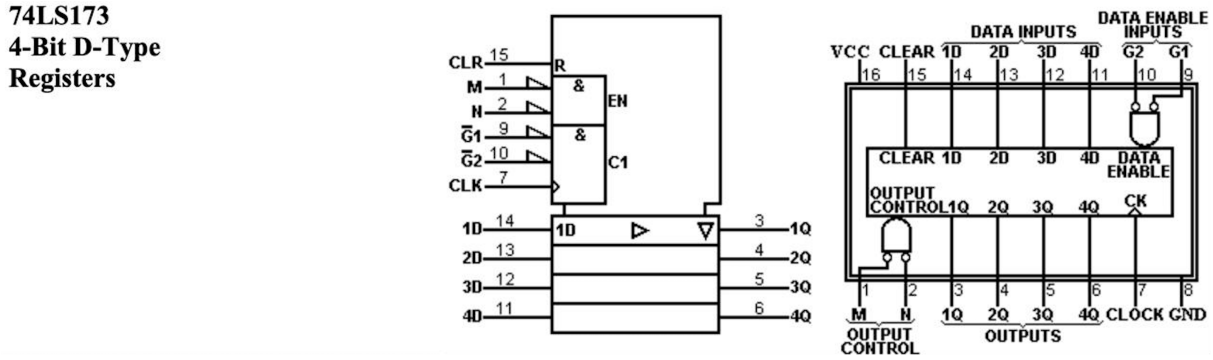

Figura 1: Diagrama funcional do registo 74LS173. Fonte: Texas Instruments

Como ilustrado na Figura 4, o 74LS173 possui:

- Entradas de dados paralelos (1D a 4D),
- Saídas sincronizadas (1Q a 4Q),
- Entradas de controlo de escrita (`G1`, `G2`),
- Sinais de controlo de ativação de saída (`M`, `N`).

No projeto, os 74LS173 foram utilizados como:

- R_X – registo de entrada dos operandos,
- R_A – registo acumulador, que armazena o resultado da ALU,
- R_F – registo de flags, que guarda o estado do resultado (carry, overflow, sinal, zero).

4.3 Memória de Registos – 74LS670

O 74LS670 é um circuito integrado que funciona como uma memória de registos configurada como uma tabela 4x4: quatro registos de 4 bits cada.

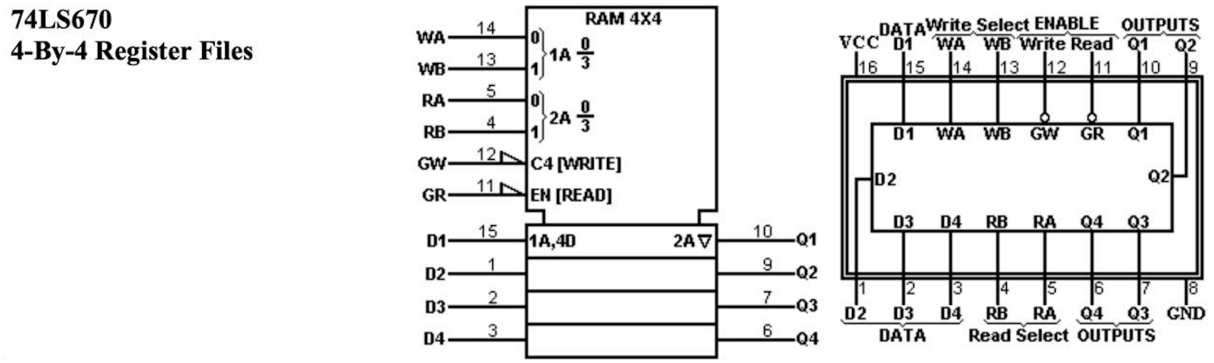

Figura 2: Diagrama funcional do 74LS670 (4x4 Register File). Fonte: Texas Instruments

Como mostrado na Figura 2, este componente inclui:

- Entradas de dados (D1 a D4),
- Sinais de controlo de escrita (GW, WA, WB),
- Sinais de controlo de leitura (GR, RA, RB),
- Saídas de leitura (Q1 a Q4).

No sistema, esta memória atua como fonte dos operandos A e B para a ALU, permitindo selecionar registos com base em dois bits de endereçamento (D_0, D_1), que definem qual registo será lido ou escrito.

4.4 Unidade Lógica Aritmética (ALU)

A ALU é o bloco responsável pela execução das operações aritméticas. Foi implementada numa GAL22V10, recorrendo a lógica combinatória programada para realizar as seguintes operações:

- Adição $(A + B)$,
- Subtração $(A - B)$,
- Incremento $(A + 1)$,
- Decremento $(A - 1)$.

A operação é selecionada pelos sinais de controlo M_1 e M_0.

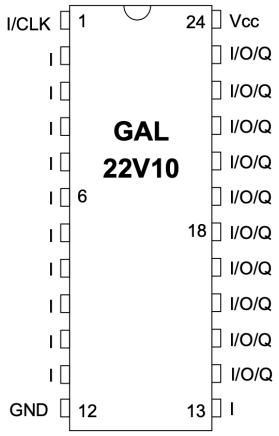

Figura 3: Diagrama funcional da gal22v10 . Fonte: Lattice Semiconductor Corp.

4.5 Registo de Flags – 74LS173

O registo das flags tem como função indicar o estado do resultado de uma operação aritmética. São consideradas Flags como Zero (Z), Sinal (S), Carry (C) e Overflow (V), comuns em arquiteturas de microprocessadores.

As flags de resultado (Zero, Sinal, Carry e Overflow) são calculadas a partir da saída da ALU e armazenadas num registo 74LS173 dedicado. O carregamento das flags é feito com o sinal W_F. Estas flags permitem identificar o estado do resultado e são importantes para a tomada de decisões em arquiteturas com desvio ou instruções condicionais.

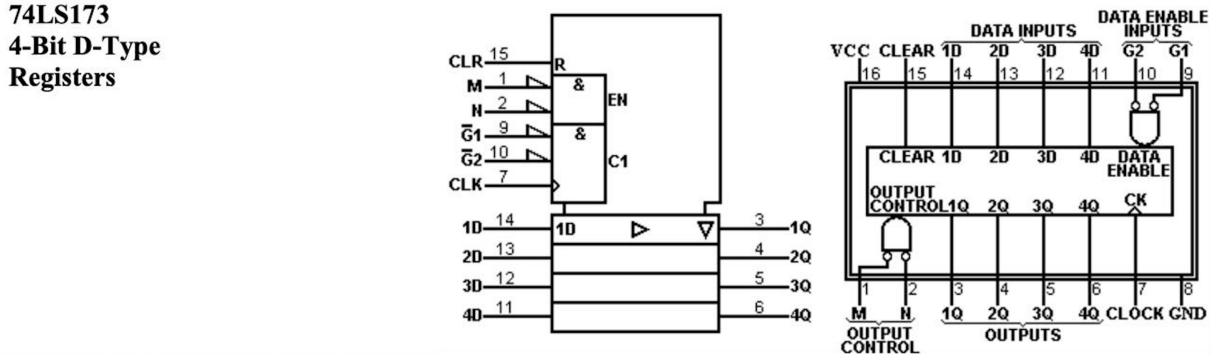

Figura 4: Diagrama funcional do registo 74LS173. Fonte: Texas Instruments

4.6 Unidade de Controlo – GAL22V10

A unidade de controlo é responsável pela ativação coordenada dos sinais de leitura e escrita nos diferentes blocos, bem como pela definição do fluxo de dados. Esta unidade foi implementada como uma máquina de estados finitos, programada numa GAL22V10, e segue uma sequência lógica baseada num fluxograma definido para o ciclo de execução das operações. Entre os sinais gerados encontram-se G_W e G_R (escrita e leitura global), D_0 e D_1 (seleção de registo), e os sinais de controlo individuais para escrita nos registos (W_X, W_F, W_{Acc}). Cada estado corresponde a uma fase do ciclo de execução (carregamento de operandos, operação, armazenamento do resultado, atualização das flags).

A unidade de controlo foi desenvolvida como uma máquina de estados finitos e programada na GAL22V10. Esta unidade gera todos os sinais que coordenam o funcionamento do sistema:

- Escrita: W_X, W_F, W_{Acc}, GW
- Leitura: GR
- Endereçamento: D_0, D_1
- Seleção de operação ALU: M_1, M_0

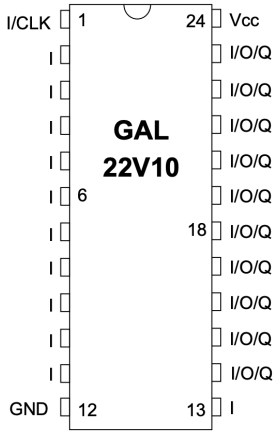

Figura 5: Diagrama funcional da gal22v10 . Fonte: Lattice Semiconductor Corp.

5 Especificação do Projeto

O projeto desenvolvido tem como objetivo a conceção e implementação, em hardware, de uma arquitetura, composta por uma Unidade Lógica Aritmética (ALU), Registos de Dados, Registo de Flags, uma Memória de Registos, e uma Unidade de Controlo . A arquitetura foi concebida para executar um conjunto restrito de operações aritméticas, com suporte à sinalização de condições de resultado através de flags.

A especificação do sistema foi orientada para uma estrutura modular, facilitando a análise e implementação em blocos funcionais distintos. Esta modularidade permite o reuso e a escalabilidade da arquitetura, mantendo a simplicidade do controlo e a clareza das ligações entre módulos.

5.1 Objetivos Funcionais

O sistema deve ser capaz de:

- Carregar operandos a partir de uma memória de registos (74LS670);
- Executar uma operação aritmética entre dois operandos: $A+B$, $A-B$, $A+1$, $A-1$;
- Armazenar o resultado da operação num registo acumulador;
- Atualizar um conjunto de flags com base no resultado (Zero, Carry, Sinal, Overflow);
- Permitir o controlo sequencial de todas as fases de execução, por meio de uma máquina de estados finitos.

5.2 Componentes Principais

Os principais blocos funcionais e respetivos componentes físicos utilizados são:

- **Registos de dados:** 3 registos 74LS173 — utilizados como registo de entrada (R_X), acumulador (R_A) e registo de flags (R_F);
- **Memória de registos:** 74LS670 — permite o armazenamento e seleção de múltiplos operandos;
- **ALU:** Lógica combinatória implementada numa GAL22V10 — realiza as quatro operações aritméticas possíveis;
- **Unidade de controlo:** Máquina de estados finitos programada numa GAL22V10 — gere todos os sinais de leitura, escrita e seleção de operação.

5.3 Sinais de Controlo

Para o correto funcionamento da arquitetura, foram definidos os seguintes sinais de controlo:

- W_X, W_{Acc}, W_F — escrita nos registos R_X, acumulador e flags;
- GR, GW — leitura e escrita globais na memória 74LS670;
- RA, RB, WA, WB — seleção de registo de origem e destino na memória;
- M_1, M_0 — operação a realizar na ALU;
- D_0, D_1 — bits de endereçamento para a memória de registos.

5.4 Requisitos de Implementação

A Tabela 1 apresenta o conjunto de operações que o sistema deve implementar, com a respetiva codificação binária e descrição funcional.

Tabela 1: Requisitos de Implementação

X_3	X_2	X_1	X_0	**Operação**
0	0	1	0	$R_i + 1 \rightarrow R_i$
0	1	0	0	$R_{2i+1}R_{2i} + R_{2i+1}R_{2i} \rightarrow R_{2i+1}R_{2i}$
0	1	1	1	$R_{2i+1} + R_{2i} - 1 \rightarrow R_{2i+1}R_{2i}$
1	0	1	0	$R_i + 2 \rightarrow R_i$
0	0	0	1	$R_i - R_j \rightarrow R_i$
1	1	1	1	$R_{2i+1} + R_{2i} - 2 \rightarrow R_{2i+1}R_{2i}$

5.5 Operações Definidas pelo Sistema

O sistema foi concebido para executar um conjunto específico de operações aritméticas e de manipulação de registos. Estas operações são determinadas por uma codificação de 4 bits ($X_3X_2X_1X_0$), que define o comportamento da unidade de controlo e, por consequência, da ALU e dos registos.

Cada combinação binária ativa uma sequência específica de sinais de controlo, definidos pela máquina de estados finitos programada na GAL22V10. Estas operações demonstram que, além das funções elementares da ALU, foi introduzida lógica adicional de indexação e manipulação indireta dos registos, através de esquemas como R_{2i} e R_{2i+1}, assegurando flexibilidade computacional num sistema de arquitetura simplificada.

6 Arquitetura Implementada

A arquitetura desenvolvida neste projeto foi concebida com base em módulos digitais síncronos interligados por um barramento de 4 bits. Esta organização permite realizar operações aritméticas entre operandos armazenados em registos e guardar os resultados para posterior utilização. Toda a coordenação é feita através de uma unidade de controlo sequencial, programada numa GAL22V10.

6.1 Componentes Utilizados

A implementação em hardware recorreu a circuitos integrados TTL da família 74LS e dispositivos programáveis GAL, destacando-se os seguintes:

- **74LS173** – Utilizado como:
  - Registo de entrada de operandos (R_X);
  - Registo acumulador (R_A);
  - Registo de flags (R_F), com cada bit representando uma flag: zero (Z), carry (C), sinal (S), overflow (V).
- **74LS670** – Memória de registos 4x4, utilizada para armazenar múltiplos operandos. Possui portas dedicadas de leitura e escrita, controladas pelos sinais GR, GW, RA, RB, WA e WB.
- **GAL22V10** – Utilizada para:
  - Implementação da ALU, responsável por executar as operações aritméticas;
  - Implementação da unidade de controlo, responsável por gerar os sinais de temporização e coordenação do sistema, com base numa máquina de estados finitos.

6.2 Esquema da Arquitetura

Os dados são lidos da memória 74LS670 e transferidos para o registo R_X, sendo em seguida processados pela ALU. O resultado é guardado em R_A e as flags associadas são atualizadas em R_F.

Na Figura 6, observa-se a disposição dos principais blocos funcionais e o fluxo de dados entre eles. A ALU recebe as entradas A e B a partir do registo R_X e da memória, processa a operação definida pelos sinais M_1 e M_0 e disponibiliza o resultado para os registos seguintes.

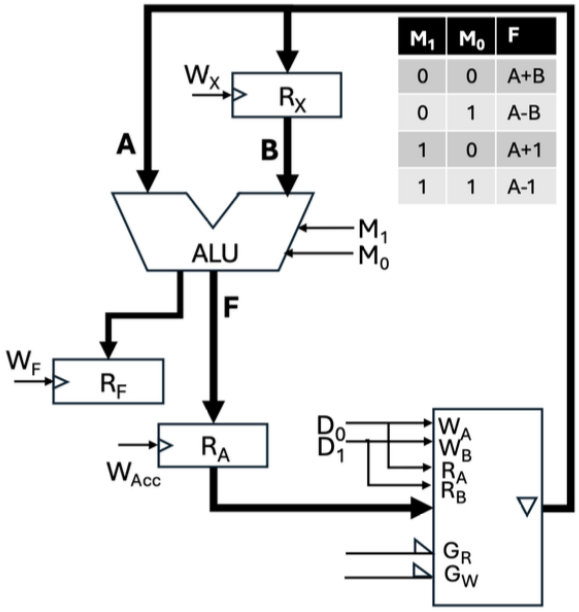

Figura 6: Esquema da arquitetura implementada.

6.3 Ligação entre Módulos

A interligação entre os diversos módulos da arquitetura é feita por meio de um barramento interno de 4 bits, que transporta os dados entre a memória, os registos e a ALU. A coordenação do acesso ao barramento e à escrita/leitura nos registos é assegurada pela unidade de controlo.

- **Leitura de operandos:** Através dos sinais GR, RA, RB, os dados são extraídos da 74LS670 e enviados para R_X via W_X.
- **Execução de operações:** A ALU realiza a operação selecionada por M_1 e M_0, utilizando os dados em R_X como entrada.
- **Armazenamento:** O resultado da ALU é armazenado em R_A por ativação de W_{Acc} e as flags são atualizadas em R_F com W_F.
- **Escrita opcional em memória:** Caso a operação o exija, o resultado pode ser reintroduzido na 74LS670 através de GW, com endereçamento pelos sinais WA ou WB.

7 Controlador

A coordenação do funcionamento da arquitetura é assegurada por um controlador implementado numa GAL22V10. Este controlador ativa os sinais necessários à leitura, execução da operação e escrita dos dados nos registos e memória.

A transição entre estados é determinada por sinais de controlo globais e, em algumas situações, pelo estado das flags de resultado, permitindo implementar decisões condicionais ou sequências de operações com diferentes durações.

7.1 Fluxograma

O comportamento do controlador foi definido a partir de um fluxograma que representa as fases principais do ciclo de execução. Cada estado do fluxograma corresponde a um estado interno da FSM programada na GAL. A sequência de estados garante que todas as etapas da operação são cumpridas de forma síncrona e coordenada.

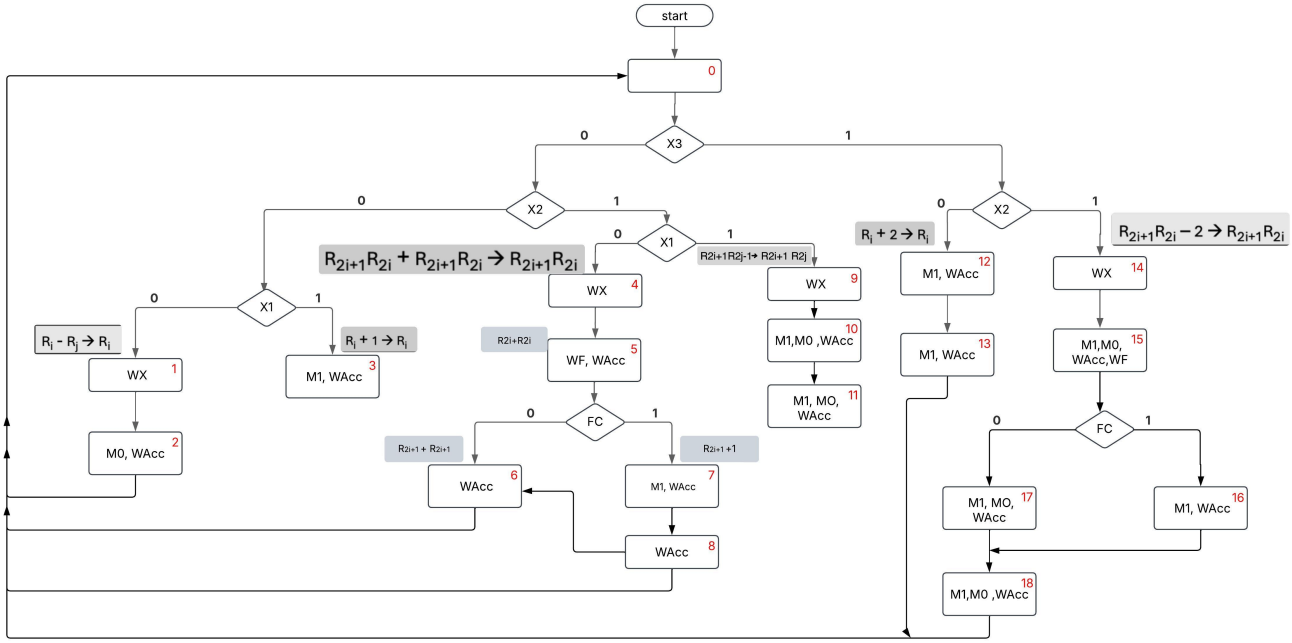

Figura 7: Fluxograma da máquina de estados do controlador digital.

A Figura 7 representa o fluxo básico do controlador digital, com os seguintes passos típicos:

1. **Estado de Início (RESET):** Inicialização do sistema, limpeza dos registos (se necessário).
2. **Leitura de operandos:** Ativação dos sinais GR, RA, RB e escrita no registo R_X com W_X.
3. **Seleção da operação:** Definida pelos bits M_1 e M_0, baseando-se na instrução recebida.
4. **Execução da operação:** A ALU processa os dados. O resultado fica disponível na saída F.
5. **Armazenamento do resultado:** Ativação de W_{Acc} para guardar o resultado em R_A.
6. **Atualização das flags:** Ativação de W_F para armazenar o estado do resultado em R_F.
7. **Escrita em memória (opcional):** Em algumas instruções, o resultado pode ser reescrito na 74LS670 com GW.
8. **Regresso ao estado de espera/início.**

A lógica foi escrita em WinCupl e sintetizada para a GAL22V10.

7.2 Equações Booleanas

Expressões Booleanas da ALU

A ALU foi implementada através de lógica combinatória, recorrendo a expressões booleanas que definem a operação a realizar com base nos sinais de controlo M_1 e M_0. A seguir apresentam-se e explicam-se as principais expressões utilizadas na construção da ALU.

- **Transformação condicional dos operandos B:**

$$B0_{out} = (B_0 \wedge \neg M_1) \oplus M_0$$
$$B1_{out} = (B_1 \wedge \neg M_1) \oplus M_0$$
$$B2_{out} = (B_2 \wedge \neg M_1) \oplus M_0$$
$$B3_{out} = (B_3 \wedge \neg M_1) \oplus M_0$$

  Estas expressões modificam dinamicamente os bits do operando B, com base nos sinais de controlo. Em particular: - Para operações de subtração ($M_1 = 1$, $M_0 = 0$), aplica-se o complemento de dois a B; - Para incremento ou decremento, gera-se um padrão fixo.

- **Cálculo do carry inicial:**

$$C_0 = M_1 \oplus M_0$$

  O primeiro carry (C_0) é gerado consoante a operação desejada. Serve para simular o bit adicional necessário em operações de complemento de dois (ex.: $A + \bar{B} + 1$).

- **Cálculo da propagação de carry:**

$$C_1 = (A_0 \wedge B0_{out}) \vee (A_0 \wedge C_0) \vee (B0_{out} \wedge C_0)$$
$$C_2 = (A_1 \wedge B1_{out}) \vee (A_1 \wedge C_1) \vee (B1_{out} \wedge C_1)$$
$$C_3 = (A_2 \wedge B2_{out}) \vee (A_2 \wedge C_2) \vee (B2_{out} \wedge C_2)$$
$$C_4 = (A_3 \wedge B3_{out}) \vee (A_3 \wedge C_3) \vee (B3_{out} \wedge C_3)$$

  Estes termos implementam um somador completo (full adder) de forma bit a bit, garantindo a propagação do carry entre os bits menos e mais significativos.

- **Cálculo do resultado:**

$$F_0 = A_0 \oplus B0_{out} \oplus C_0$$
$$F_1 = A_1 \oplus B1_{out} \oplus C_1$$
$$F_2 = A_2 \oplus B2_{out} \oplus C_2$$
$$F_3 = A_3 \oplus B3_{out} \oplus C_3$$

  Cada bit do resultado final é calculado por soma com carry (soma de três bits). O operador XOR implementa a função de soma de 1 bit com propagação de carry.

- **Cálculo das flags:**

$$FC = C_4 \quad \text{(Carry de saída)}$$
$$FOV = C_4 \oplus C_3 \quad \text{(Overflow em complemento de dois)}$$
$$FZ = \neg(F_3 \vee F_2 \vee F_1 \vee F_0) \quad \text{(Resultado igual a zero)}$$

Estas expressões garantem que a ALU realiza corretamente as operações de adição, subtração, incremento e decremento com operandos de 4 bits, e gera corretamente as flags essenciais para controlo condicional.

Expressões Booleanas de Controlo

As seguintes expressões booleanas foram utilizadas para gerar sinais de controlo específicos com base nos estados atuais da máquina de estados, codificados nos bits Q_4 a Q_0. Estas expressões foram implementadas em lógica combinatória no ficheiro de programação WinCupl da GAL22V10.

Sinais de seleção da ALU (M_1 e M_0): As expressões que determinam a operação a ser realizada pela ALU baseiam-se nos estados da FSM. Cada combinação de bits Q_i ativa uma operação específica.

- $M_1 = (!Q_0 \wedge \neg Q_2 \wedge \neg Q_3 \wedge Q_4) \vee (Q_0 \wedge Q_1 \wedge Q_3 \wedge \neg Q_4) \vee (\neg Q_0 \wedge Q_1 \wedge \neg Q_2 \wedge \neg Q_4)$

  Esta expressão ativa M_1 em três situações distintas, associadas a diferentes instruções aritméticas como subtração ou decremento. Cada termo corresponde a uma configuração específica dos bits de estado.

- $M_0 = (!Q_1 \wedge Q_2 \wedge Q_3 \wedge \neg Q_4) \vee (\neg Q_0 \wedge \neg Q_2 \wedge \neg Q_3 \wedge Q_4) \vee (Q_1 \wedge \neg Q_2 \wedge Q_3 \wedge \neg Q_4) \vee (\neg Q_1 \wedge \neg Q_2 \wedge \neg Q_3 \wedge Q_4) \vee (Q_0 \wedge Q_1 \wedge \neg Q_4)$

O sinal M_0 apresenta mais combinações, o que indica que é utilizado em múltiplas instruções. Juntamente com M_1, define-se a operação exata executada pela ALU (por exemplo: soma, subtração, incremento ou decremento).

Sinal de controlo de escrita no endereço (WA_CTRL):

- $WA_CTRL = (\neg Q_3 \wedge Q_0) \vee (\neg Q_3 \wedge Q_2) \vee (Q_3 \wedge \neg Q_2 \wedge \neg Q_0) \vee (Q_2 \wedge Q_0) \vee (\neg Q_2 \wedge Q_1)$

  Esta expressão determina se o endereço da memória de registos 74LS670 deve ser ativado para escrita. Cada termo representa uma condição em que o sistema deverá armazenar o resultado da operação num dos registos da memória.

Sinal de controlo de escrita no registo de entrada (WX_CTRL):

- $WX_CTRL = (Q_3 \wedge \neg Q_2 \wedge \neg Q_1 \wedge Q_0) \vee (Q_3 \wedge Q_2 \wedge \neg Q_0)$

  Esta expressão ativa a escrita no registo R_X, que armazena temporariamente os operandos lidos da memória. A ativação ocorre em dois estados distintos da FSM.

Sinal de controlo de escrita global (GW_CTRL):

- $GW_CTRL = WA_CTRL$

  Neste caso, o sinal de escrita global na memória 74LS670 (GW) é definido com base na mesma lógica que controla a escrita do endereço. Isso assegura que a ativação da escrita ocorre de forma sincronizada com a seleção do destino na memória.

7.3 Código fonte em WinCupl para a GAL22V10

Implementação da ALU em WinCupl

A ALU foi implementada numa GAL22V10, utilizando o ambiente WinCupl. Aceita dois operandos de 4 bits ($A[3..0]$, $B[3..0]$) e realiza diferentes operações aritméticas consoante os sinais de controlo M_1 e M_0. A operação é realizada com lógica combinatória e suporta a propagação de carry bit a bit, permitindo detetar carry global e overflow.

Código WinCupl da ALU:

```
B0out = (B0 & !M1) $ M0;
B1out = (B1 & !M1) $ M0;
B2out = (B2 & !M1) $ M0;
B3out = (B3 & !M1) $ M0;
C0 = M1 $ M0;
C1 = A0 & B0out # A0 & C0 # B0out & C0;
C2 = A1 & B1out # A1 & C1 # B1out & C1;
```

```
C3 = A2 & B2out # A2 & C2 # B2out & C2;
C4 = A3 & B3out # A3 & C3 # B3out & C3;

F3 = A3 $ B3out $ C3;
F2 = A2 $ B2out $ C2;
F1 = A1 $ B1out $ C1;
F0 = A0 $ B0out $ C0;

FOV = C4 $ C3;
FC = C4;
FZ = !(F3 # F2 # F1 # F0);
```

Descrição funcional:

- **Sinais de controlo:** Os sinais $M1$ e $M0$ determinam a operação aritmética:
  - `M1`, M0 - 00 — Soma $A + B$
  - `M1`, M0 - 01 — Soma $A + 1$ (incremento)
  - `M1`, M0 - 10 — Subtração $A - B$
  - `M1`, M0 -11 — Subtração $A - 1$ (decremento)
- **Bloco B0out–B3out:** Gera os bits modificados de B, que são utilizados consoante a operação desejada: - Para somas e subtrações, aplica-se uma inversão condicional de bits, simulando o complemento de dois.
- **Propagação de carry:** Cálculo sequencial do carry ($C0$–$C4$) com base em somadores de 1 bit com lógica combinatória. O resultado depende dos operandos e carry anterior.
- **Resultado final:** Os bits de saída $F[3..0]$ são obtidos com a operação $A \oplus B \oplus C_n$.
- **Flags:**
  - `FZ` — Flag Zero: Ativada quando o resultado é zero.
  - `FC` — Flag Carry: Indica carry de saída (bit $C4$).
  - `FOV` — Flag Overflow: Detetado por XOR entre $C4$ e $C3$, típico em soma/subtração de inteiros com sinal.

Controlador:

O controlador digital foi implementado e programado numa GAL22V10 utilizando a linguagem WinCupl. O excerto abaixo mostra a definição dos estados, condições de transição e expressões lógicas para gerar os sinais de controlo.

```
Name     Name ;
PartNo   00 ;
Date     14/03/2025 ;
Revision 01 ;
Designer Engineer ;
Company  ubi ;
Assembly None ;
Location  ;
Device   g22v10 ;

/* *************** INPUT PINS *********************/
PIN  1   = CLK                        ;
PIN  2   = X3                         ;
PIN  3   = X2                         ;
PIN  4   = X1                         ;
PIN  5   = X0                         ;
PIN  6   = R                          ;
PIN  7   = INIC                       ;
PIN  8   = Fc                         ;
PIN  9   = I                          ;
PIN  10  = J                          ;

/* *************** OUTPUT PINS *********************/
PIN 14   = M1                         ;
PIN 15   = M0                         ;
PIN 16   = WF                         ;
PIN 17   = Q0                         ;
PIN 20   = Q1                         ;
PIN 21   = Q2                         ;
PIN 22   = Q3                         ;
PIN 23   = Q4                         ;

FIELD estado = [Q4..0];

$DEFINE EST0  'b'00000
```

```
$DEFINE EST1  'b'00001
$DEFINE EST2  'b'00010
$DEFINE EST3  'b'00011
$DEFINE EST4  'b'00100
$DEFINE EST5  'b'00101
$DEFINE EST6  'b'00110
$DEFINE EST7  'b'00111
$DEFINE EST8  'b'01000
$DEFINE EST9  'b'01001
$DEFINE EST10 'b'01010
$DEFINE EST11 'b'01011
$DEFINE EST12 'b'01100
$DEFINE EST13 'b'01101
$DEFINE EST14 'b'01110
$DEFINE EST15 'b'01111
$DEFINE EST16 'b'10000
$DEFINE EST17 'b'10001
$DEFINE EST18 'b'10010

SEQUENCE estado

PRESENT EST0
   if(!X3&!X2&!X1) next EST1;
   if(!X3&!X2&X1)  next EST3;
   if(!X3&X2&!X1)  next EST4;
   if(!X3&X2&X1)   next EST9;
   if(!X3&!X2)     next EST12;
   if(X3&X2)       next EST14;

PRESENT EST1  next EST2;
PRESENT EST2  next EST0;
PRESENT EST3  next EST0;
PRESENT EST4  next EST5;
PRESENT EST5
   if !Fc next EST6;
   if  Fc next EST7;
PRESENT EST6  next EST0;
PRESENT EST7  next EST8;
PRESENT EST8  next EST0;
```

```
PRESENT EST9  next EST10;
PRESENT EST10 next EST11;
PRESENT EST11 next EST0;
PRESENT EST12 next EST13;
PRESENT EST13 next EST0;
PRESENT EST14 next EST15;
PRESENT EST15
   if !Fc next EST18;
   if  Fc next EST17;
PRESENT EST17 next EST18;
PRESENT EST18 next EST0;

estado.ar = !INIC;
estado.sp = 'b'0;

M1 = (!Q0 &! Q2 & !Q3 & Q4) # (Q0 & Q1 & Q3 & !Q4) # (!Q0 &Q1 &! Q2 & !Q4);
M0 = (!Q1 & Q2 & Q3 & !Q4) # (!Q0 & !Q2 & !Q3 & Q4) # (Q1 & !Q2 & Q3 & !Q4) # (!Q1 &
WA_CTRL = (!Q3 & Q0) # (!Q3 & Q2) # (Q3 & !Q2 & !Q0) # (Q2 & Q0) # (!Q2 & Q1);
WX_CTRL = (Q3 & !Q2 & !Q1 & Q0) # (Q3 & Q2 & !Q0);
GW_CTRL = WA_CTRL;

WA = !(CLK & WA_CTRL);
WF = !(CLK & WF_CTRL);
WX = !(CLK & WX_CTRL);
GW = !(!CLK & WA_CTRL);
```

8 Testes e Validação

Nesta secção descrevem-se os testes realizados à arquitetura desenvolvida, com o objetivo de validar o seu funcionamento correto à medida que os módulos foram sendo concluídos. A validação foi feita em duas fases: primeiro a ALU foi testada individualmente; em seguida, o controlador foi implementado mas ainda não validado funcionalmente de forma completa.

8.1 Testes à Unidade Lógica Aritmética (ALU)

A ALU foi testada de forma isolada, fornecendo operandos de entrada (A e B) e os sinais de controlo (M_1, M_0), e verificando os resultados de saída ($F_3..F_0$), bem como as flags (FZ, FC, FOV).

Foram testadas as seguintes operações:

- **Soma** $A + B$ com diferentes combinações de operandos, validando a propagação de carry e geração da flag de overflow;
- **Subtração** $A - B$ com operandos em complemento de dois, confirmando a correta operação da lógica de inversão e soma com $+1$;
- **Incremento** $A + 1$ e **Decremento** $A - 1$, com verificação do comportamento das flags de zero e carry;
- **Deteção de zero** (FZ) em casos como $A = B$ ou $A = 0$, confirmando a ativação correta da flag.

Todos os testes realizados demonstraram um comportamento correto da ALU, de acordo com as expressões booleanas implementadas.

8.2 Estado Atual do Controlador

O controlador, responsável pela coordenação sequencial dos sinais, já foi implementado em hardware através da GAL22V10. No entanto, até ao momento, não conseguimos realizar testes funcionais completos ao sistema com o controlador integrado.

8.3 Considerações Finais sobre os Testes

Os testes à ALU confirmaram a estabilidade e fiabilidade da operação aritmética. Apesar da ausência de testes completos ao controlador, a preparação do código e montagem do circuito garantem que a arquitetura se encontra funcionalmente estruturada.

9 Conclusão

O presente trabalho teve como objetivo a implementação de uma Arquitetura , composta por uma Unidade Lógica Aritmética (ALU), Registos de Armazenamento, Memória, e um Controlador, recorrendo a circuitos integrados da família 74LS e a dispositivos programáveis GAL22V10.

Ao longo do projeto, foi possível consolidar conhecimentos teóricos sobre sistemas digitais e aprofundar competências práticas na utilização de ferramentas como o WinCupl bem como desenvolver competências na montagem de circuitos em hardware.

A ALU foi devidamente implementada e testada, tendo-se verificado o correto funcionamento das operações aritméticas e das flags associadas, como Zero, Carry e Overflow.

O Controlador foi programado, de acordo com o fluxograma, embora ainda não tenha sido possível proceder à sua validação funcional completa devido a limitações de tempo.

Durante o desenvolvimento do projeto, a gestão de tempo revelou-se um desafio, especialmente devido à complexidade da implementação do controlador e à má gestão do tempo, da nossa parte.O que dificultou a integração total do sistema pois sofreu atrasos, impedindo a validação final do controlador. Em suma, o projeto constituiu uma experiência rica em termos académicos e práticos, permitindo aplicar conceitos fundamentais de microprocessadores num contexto real de implementação.

Referências

[1] M. Morris Mano, *Computer System Architecture*, 3rd Edition, Pearson, 2007.

[2] Atmel Corporation, *WinCupl User's Manual*, disponível em: `http://www.atmel.com/`.

[3] Atmel Corporation, *GAL22V10 Programmable Logic Device Datasheet*, disponível em: `https://www.datasheetarchive.com/`.

[4] Texas Instruments, *SN74LS181 – 4-bit Arithmetic Logic Unit*, Datasheet, 1986.

[5] Texas Instruments, *SN74LS173 – Quad D-Type Register with 3-State Outputs*, Datasheet, 1986.

[6] Texas Instruments, *SN74LS670 – 4x4 Register File*, Datasheet, 1986.

[7] Universidade da Beira Interior, *Aulas Teóricas de Microprocessadores*, Prof. Doutor António M. Pinheiro, 2025. [AulaT1.pdf, AulaT3.pdf, AulaT4.pdf].